

Практическое занятие №4

Вариант №1

1. Создайте Series из последовательности 15 значений, равномерно разбивающих отрезок [0, 20] (воспользуйтесь функцией `linspace`). Определите отношение элементов полученной серии к их предыдущим элементам. В результате необходимо получить среднее полученного вектора, оставив в нём только те значения, которые не больше 1.5.

Пояснения:

- Если было бы необходимо найти последовательность из 3-х значений, равномерно разбивающих отрезок [0,1], то это были бы значения [0, 0.5, 1].
- Если был бы дан список элементов `a = [1, 2, 3, 12]`, отношения элементов к предыдущим будут равны `[NaN, 2, 1.5, 4]`, а на последнем этапе останется только [1.5] и среднее значение будет также 1.5.

2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` задайте столбец `customer_id` в качестве индекса, выделите клиента с максимальной суммой транзакции (то есть с максимальным приходом на карту) и найдите у него наиболее часто встречающийся модуль суммы приходов/ расходов.

Пояснения:

- Если у клиента были транзакции `[-10000, 10, 0, -10]`, то максимумом будет являться значение 10.
- Если у клиента были транзакции `[-10000, 10, 0, -10]`, то наиболее встречающийся модуль суммы транзакций равен 10, и встретился он 2 раза.

Вариант №2

1. По клиенту получены зашумленные данные (объект `s` типа Series) по его транзакциям:
`s = pd.Series(data=['1', 2, 3.1, 'hi', 5, -512, 12.42, 'sber', 10.10, 98], index=range(6, 26, 2))`
Для заданного объекта `s` выберите из `s` элементы с индексами 6 и 8, после чего просуммируйте их и сохраните результат.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю `amount` из таблицы `transactions` по строкам, отсортированным по полю `amount` в порядке возрастания, и из которых удалены дублирующиеся по столбцам `[mcc_code, tr_type]` строки, причём при удалении соответствующих дублей остаются только последние из дублирующихся строк (`keep='last'`).

Вариант №3

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['1', 2, 3.1, 'hi!', 5, -512, 12.42, 'sber', 10.10, 98], index=range(6, 26, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью функции из библиотеки `numpy`: `np.var( , ddof=0)`. Полученный результат округлите до 2-х знаков после запятой.
2. Считайте в переменные `tr_types`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-5 транзакций по частоте встречаемости следующие пункты:
 - Списание по требованию
 - Оплата услуги. Банкоматы СБ РФ
 - Погашение кредита (в пределах одного филиала)

Вариант №4

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['1', 2, 3.1, 'hi!', 5, -512, 12.42, 'sber', 10.10, 98], index=range(6, 26, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью встроенной в `python` функции: `.var(ddof=1)`. Полученный результат округлите до 1-го знака после запятой.
2. Считайте в переменные `tr_mcc_codes`, `tr_types`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В `tr_types` выберите произвольные 50 строк с помощью метода `sample` (указав при этом `random_seed` равный 242). В полученной подвыборке найдите долю наблюдений (столбец `tr_description`), в которой содержится подстрока 'списание' (в любом регистре). Выведите ответ в виде вещественного числа, округлённого до одного знака после запятой, отделив дробную часть точкой в формате "123.4"

Пояснение: строки "Списание с карты", "списание", "СПИСАНИЕ" удовлетворяют условию, так как будучи переведёнными в нижний регистр содержат подстроку "списание".

Вариант №5

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям: `s = pd.Series(data=['1', 2, 3.1, 'hi', 5, -512, 12.42, 'sber', 10.10, 98], index=range(6, 26, 2))`
Для заданного объекта *s* создайте новый Series, значения которого совпадают со значениями *s*, а индексы – это целочисленные значения (имеющие тип `int`) от 2 до 12, не включая 12.
2. Считайте в переменные `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` задайте столбец `customer_id` в качестве индекса, выделите клиента с минимальной суммой транзакции (то есть с минимальным приходом на карту или с максимальным расходом с карты), найдите у него наиболее часто встречающийся модуль суммы приходов/расходов и сколько раз он встречался.

Пояснения:

- Если у клиента были транзакции `[-10000, 10, 0, -10]`, то минимумом будет являться значение `-10000`.
- Если у клиента были транзакции `[-10000, 10, 0, -10]`, то наиболее встречающийся модуль суммы транзакций равен `10`, и встретился он `2` раза.

Вариант №6

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии в 3 степень, а значения индекса увеличьте в 3 раза и выведите сумму элементов, строго меньших 2.6, имеющих нечётные значения индекса.
2. Считайте в переменные `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю `amount` из таблицы `transactions` по тем строкам, которые ни в одном из своих столбцов не содержат пустые значения.

Вариант №7

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии в 3 степень, а значения индекса увеличьте в 3 раза и выведите количество значений серии меньше нуля.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_mcc_codes`, `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану, моду и среднее значение суммы транзакций по полю `amount` из таблицы `transactions`.

Вариант №8

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии во 2 степень, а значения индекса увеличьте в 3 раза и выведите сумму элементов, строго больших 2.2, имеющих чётные значения индекса.

Пояснения:

- Фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).
 - Следует внимательнее использовать `[ ]` для выбора данных по нескольким условиям: либо выбирать данные последовательно, либо сразу по нескольким условиям, но через оператор `&`. Отличие оператора `and` от оператора `&`: `and` - выводит последнее проверенное значение, `&` - выводит пересечение значений. Пример: `s[ _ & _ ].sum()`
2. Считайте в переменные `tr_mcc_codes`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
 3. В датафрейме `transactions` отсортируйте данные по возрастанию суммы транзакций, выведите из них последние 3 и найдите их среднее значение.

Вариант №9

1. Создайте Series из последовательности 20 значений, равномерно разбивающих отрезок [0, 25] (воспользуйтесь функцией `linspace`). Определите отношение элементов полученной серии к их последующим элементам. В результате необходимо получить среднее полученного вектора, оставив в нём только те значения, которые не превышают 0.85. Полученный результат округлите до трех знаков после запятой.

Пояснения:

- Если было бы необходимо найти последовательность из 3-х значений, равномерно разбивающих отрезок [0,1], то это были бы значения [0, 0.5, 1].
- Если был бы дан список элементов $a = [1.8, 2, 4, 16]$, отношения элементов к последующим будут равны [0.9, 0.5, 0.25, NaN], а на последнем этапе останется только [0.5, 0.25] и среднее значение будет 0.375.

2. Считайте в переменные `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые $n=1000000$ строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В `tr_types` выберите произвольные 100 строк с помощью метода `sample` (указав при этом `random_seed` равный 242). В полученной подвыборке найдите долю наблюдений (столбец `tr_description`), в которой содержится подстрока 'плата' (в любом регистре). Выведите ответ в виде вещественного числа, округлённого до двух знаков после запятой, отделив дробную часть точкой в формате "123.45"

Пояснение: строки "ПлатА за аренду", "ПлатАза аренду", "ПЛАТА" удовлетворяют условию, так как будучи переведёнными в нижний регистр содержат подстроку "плата".

Вариант №10

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии в 3 степень, а значения индекса увеличьте в 2 раза и выведите количество значений серии больше нуля.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые $n=1000000$ строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте (выведите) содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-5 транзакций по частоте встречаемости следующие пункты:
 - Выдача наличных в АТМ Сбербанк России.
 - Комиссия за обслуживание ссудного счета.

Вариант №11

1. Создайте Series из последовательности 25 значений, равномерно разбивающих отрезок [0, 20] (воспользуйтесь функцией `linspace`). Определите количество получившихся элементов, которые являются нечетными числами, и найдите их сумму.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` задайте столбец `customer_id` в качестве индекса, выделите клиента с максимальной по модулю суммой транзакции и найдите у него наиболее часто встречающийся модуль суммы приходов/расходов.

Пояснения:

- Если у клиента были транзакции `[-500, 10, 0, -10]`, то максимумом по модулю будет являться значение 500.
- Если у клиента были транзакции `[-500, 10, 0, -10]`, то наиболее встречающийся модуль суммы транзакций равен 10, и встретился он 2 раза.

Вариант №12

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Умножьте каждое значение серии на 3, а значения индекса – на 2, и выведите количество положительных значений серии с индексом не более 150.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_mcc_codes`, `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану и среднее значение суммы транзакций по первым 100 строкам, отсортированным по полю `amount` из таблицы `transactions` в порядке убывания.

Вариант №13

1. Создайте Series из последовательности 8 значений, равномерно разбивающих отрезок [0, 9] (воспользуйтесь функцией `linspace`). Определите количество получившихся элементов, которые являются четными числами, и найдите сумму элементов, которые являются нечетными числами.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-5 транзакций по частоте встречаемости следующие пункты:
 - Оплата услуги. Банкоматы СБ РФ
 - Погашение кредита (в пределах одного филиала)
 - Списание по требованию

Вариант №14

1. По клиенту получены зашумленные данные (объект `s1` типа Series) по его транзакциям:
`s1 = pd.Series(data=['1', 2, 3.1, 'hi!', 15, -52, 12.42, 'sb', 10.1, 98], index=range(6, 26, 2))`
Выберите из `s1` элементы с индексами 8 и 14, сложите их, из полученного результата вычтите количество целочисленных элементов (имеющих тип `int`), ответ сохраните в переменную `s2`.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю `amount` из таблицы `transactions` по строкам, отсортированным по полю `amount` в порядке убывания, и из которых удалены дублирующиеся по столбцам `[mcc_code, tr_type]` строки, причём при удалении соответствующих дублей остаются только последние из дублирующихся строк (`keep='last'`).

Вариант №15

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['5', 7, 3.7, 'Hi!', 5, -517, 12.42, 'ss5', 104.2, 945, 235], index=range(4, 26, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью функции из библиотеки `numpy`: `np.var( , ddof=0)`. Полученный результат округлите до 2 знаков после запятой.
2. Считайте в переменные `tr_types`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-5 транзакций по частоте встречаемости следующие пункты:
 - Списание по требованию
 - Покупка. POS ТУ СБ РФ
 - Плата за перевод на карту (с карты) через Мобильный банк

Вариант №16

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['4', 2, 3.1, 'hi!', 5, -512, 12.42, 's123', 10.10, 98], index=range(6, 26, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью встроенной в `python` функции: `.var(ddof=1)`. Полученный результат округлите до целого числа.
2. Считайте в переменные `tr_mcc_codes`, `tr_types`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В `tr_types` выберите произвольные 25 строк с помощью метода `sample` (указав при этом `random_seed` равный 242). В полученной подвыборке найдите долю наблюдений (столбец `tr_description`), в которой содержится подстрока 'списание' (в любом регистре). Выведите ответ в виде вещественного числа, округлённого до одного знака после запятой, отделив дробную часть точкой в формате "123.4"

Пояснение: строки "Списание с карты", "списание", "СПИСАНИЕ" удовлетворяют условию, так как будучи переведёнными в нижний регистр содержат подстроку "списание".

Вариант №17

1. Сгенерируйте Series из 120 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии во 3 степень, а значения индекса увеличьте в 3 раза и выведите количество значений серии больше нуля.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте (выведите) содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-5 транзакций по частоте встречаемости следующие пункты:
 - Выдача наличных в АТМ Сбербанк России.
 - Комиссия за обслуживание ссудного счета.
 - Покупка. ТУ Россия.

Вариант №18

1. Создайте Series из последовательности 30 значений, равномерно разбивающих отрезок `[0, 20]` (воспользуйтесь функцией `linspace`). Определите количество получившихся элементов, которые являются нечетными числами, и найдите их сумму.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` задайте столбец `customer_id` в качестве индекса, выделите клиента с максимальной по модулю суммой транзакции и найдите у него наиболее часто встречающийся модуль суммы приходов/ расходов.

Пояснения:

- Если у клиента были транзакции `[-500, 10, 0, -10]`, то максимумом по модулю будет являться значение 500.
- Если у клиента были транзакции `[-500, 10, 0, -10]`, то наиболее встречающийся модуль суммы транзакций равен 10, и встретился он 2 раза.

Вариант №19

1. Сгенерируйте Series из 150 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`

Умножьте каждое значение серии на 3, а значения индекса – на 2, и выведите количество положительных значений серии с индексом не более 150.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_mcc_codes`, `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану и среднее значение суммы транзакций по первым 150 строкам, отсортированным по полю `amount` из таблицы `transactions` в порядке убывания.

Вариант №20

1. Создайте Series из последовательности 8 значений, равномерно разбивающих отрезок `[0, 9]` (воспользуйтесь функцией `linspace`). Определите количество получившихся элементов, которые являются четными числами, и найдите сумму элементов, которые являются нечетными числами.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для поля `tr_type` датафрейма `transactions` посчитайте частоту встречаемости всех типов транзакций `tr_type` в `transactions`. Проверьте попали ли в топ-7 транзакций по частоте встречаемости следующие пункты:
 - Плата за перевод на карту (с карты) через Мобильный банк.
 - Выдача наличных в АТМ Сбербанка (в других ТБ).
 - Внос наличных через АТМ (в своем тер.банке).

Вариант №21

1. По клиенту получены зашумленные данные (объект *s1* типа Series) по его транзакциям:
`s1 = pd.Series(data=['7', 4, 8.3, 'hi!', 15, -52, 12.42, 's123', 120.1, 58], index=range(6, 26, 2))`
Выберите из *s1* элементы с индексами 8 и 14, сложите их, из полученного результата вычтите количество целочисленных элементов (имеющих тип `int`), ответ сохраните в переменную *s2*.
2. Считайте в переменные *tr_mcc_codes* и *transactions* из одноимённых таблиц из папки *data*. Для таблицы *transactions* используйте только первые *n=500000* строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю *amount* из таблицы *transactions* по строкам, отсортированным по полю *amount* в порядке убывания, и из которых удалены дублирующие по столбцам [*mcc_code*, *tr_type*] строки, причём при удалении соответствующих дублей остаются только последние из дублирующихся строк (`keep='last'`).

Вариант №22

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['1', 2, 3.1, 'int', 5, -554, 12.42, 'str', 14.2, 245, 25], index=range(4, 26, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью функции из библиотеки *numpy*: `np.var( , ddof=0)`. Полученный результат округлите до 3 знаков после запятой.
2. Считайте в переменные *tr_types*, *transactions* и *gender_train* из одноимённых таблиц из папки *data*. Для таблицы *transactions* используйте только первые *n=1000000* строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для поля *tr_type* датафрейма *transactions* посчитайте частоту встречаемости всех типов транзакций *tr_type* в *transactions*. Проверьте попали ли в топ-10 транзакций по частоте встречаемости следующие пункты:
 - Перевод на карту (с карты) через Мобильный банк (с взиманием комиссии с отправителя)
 - Покупка. Зарубеж. ТУ
 - Списание с карты по операции “перевода с карты на карту” через АТМ (в пределах одного тер.банка)

Вариант №23

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям:
`s = pd.Series(data=['9', 3, 7.1, 'hi!', 8, -52, 12.42, 's123', 15.13, 92], index=range(8, 28, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их дисперсию с помощью встроенной в python функции: `.var(ddof=1)`. Полученный результат округлите до целого числа.
2. Считайте в переменные `tr_mcc_codes`, `tr_types`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В `tr_types` выберите произвольные 25 строк с помощью метода `sample` (указав при этом `random_seed` равный 242). В полученной подвыборке найдите долю наблюдений (столбец `tr_description`), в которой содержится подстрока 'списание' (в любом регистре). Выведите ответ в виде вещественного числа, округлённого до одного знака после запятой, отделив дробную часть точкой в формате "123.4"

Пояснение: строки "Списание с карты", "списание", "СПИСАНИЕ" удовлетворяют условию, так как будучи переведёнными в нижний регистр содержат подстроку "списание".

Вариант №24

1. По клиенту получены зашумленные данные (объект *s1* типа Series) по его транзакциям:
`s1 = pd.Series(data=['12', 9, 1.1, 'list', 15, -52, 12.42, 'set', 10.1, 98], index=range(6, 26, 2))`
Выберите из *s1* элементы с индексами 8 и 14, сложите их, из полученного результата вычтите количество целочисленных элементов (имеющих тип `int`), ответ сохраните в переменную *s2*.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=500000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю `amount` из таблицы `transactions` по строкам, отсортированным по полю `amount` в порядке убывания, и из которых удалены дублирующиеся по столбцам `[mcc_code, tr_type]` строки, причём при удалении соответствующих дублей остаются только последние из дублирующихся строк (`keep='last'`).

Вариант №25

1. Сгенерируйте Series из 140 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии в 3 степень, а значения индекса увеличьте в 4 раза и выведите количество значений серии меньше нуля.

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `tr_mcc_codes`, `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану, моду и среднее значение суммы транзакций по полю `amount` из таблицы `transactions`.

Вариант №26

1. Сгенерируйте Series из 150 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя: `np.random.seed(242)`
Возведите каждое значение серии во 2 степень, а значения индекса увеличьте в 3 раза и выведите сумму элементов, строго больших 2.2, имеющих чётные значения индекса.

Пояснения:

- Фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).
- Следует внимательнее использовать `[ ]` для выбора данных по нескольким условиям: либо выбирать данные последовательно, либо сразу по нескольким условиям, но через оператор `&`. Отличие оператора `and` от оператора `&`: `and` - выводит последнее проверенное значение, `&` - выводит пересечение значений. Пример: `s[ _ & _ ].sum()`

2. Считайте в переменные `tr_mcc_codes`, `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` отсортируйте данные по возрастанию суммы транзакций, выведите из них последние 5 и найдите их среднее значение.

Вариант №27

1. Создайте Series из последовательности 12 значений, равномерно разбивающих отрезок [0, 24] (воспользуйтесь функцией `linspace`). Определите отношение элементов полученной серии к их предыдущим элементам. В результате необходимо получить среднее полученного вектора, оставив в нём только те значения, которые превышают 0.8. Полученный результат округлите до трёх знаков после запятой.

Пояснения:

- а. Если бы было необходимо найти последовательность из 4-х значений, равномерно разбивающих отрезок [0,2], то это были бы значения [0, 0.6667, 1.3333, 2].
 - б. Если дан список элементов $a = [2, 4, 8, 5]$, отношения элементов к последующим будут равны [0.5, 0.5, 1.6, NaN], а на последнем этапе останется [0.5, 0.5] и среднее будет 0.5.
2. Считайте в переменные `tr_mcc_codes` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые $n=1000000$ строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
 3. В датафрейме `transactions` задайте столбец `customer_id` в качестве индекса, выделите клиента с максимальной суммой транзакции (то есть с максимальным приходом на карту) и найдите у него наиболее часто встречающийся модуль суммы приходов/ расходов.

Пояснения:

- Если у клиента были транзакции [-10000, 10, 0, -10], то максимумом будет являться значение 10.
- Если у клиента были транзакции [-10000, 10, 0, -10], то наиболее встречающийся модуль суммы транзакций равен 10, и встретился он 2 раза.

Вариант №28

1. По клиенту получены зашумленные данные (объект `s` типа Series) по его транзакциям:
`s = pd.Series(data=['a', 2, 3.5, 'b', 6, -10, 0.5, 'c', 8, 4], index=range(5, 25, 2))`
Для заданного объекта `s` выберите из `s` элементы с индексами 7 и 13, сложите их, из полученного результата вычтите количество элементов, имеющих строковый тип (`str`). Ответ сохраните в переменную `result`.
2. Считайте в переменные `tr_mcc_codes`, `tr_types` и `transactions` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые $n=500000$ строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю `amount` из таблицы `transactions` по строкам, отсортированным по полю `amount` в порядке возрастания, и из которых удалены дублирующиеся по столбцам `[mcc_code, tr_type]` строки, причём при удалении соответствующих дублей остаются только первые из дублирующихся строк (`keep='first'`).

Вариант №29

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям: `s = pd.Series(data=[1, 2.2, 'text', 4, 5.5, 6, '7', 8.8, 9, 10], index=range(0, 20, 2))`
Выберите из *s* только элементы, имеющие числовой тип, и вычислите их дисперсию с помощью встроенной функции `.var(ddof=1)`. Полученный результат округлите до двух знаков после запятой.
2. Считайте в переменные *tr_types*, *transactions* и *gender_train* из одноимённых таблиц из папки *data*. Для таблицы *transactions* используйте только первые *n=1000000* строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В таблице *tr_types* выберите произвольные 50 строк с помощью метода `sample` (указав `random_seed=242`). В полученной подвыборке найдите долю наблюдений (столбец *tr_description*), в которых содержится подстрока 'кредит' (в любом регистре). Выведите ответ в виде вещественного числа, округлённого до двух знаков после запятой, отделив дробную часть точкой в формате "0.xx".

Вариант №30

1. По клиенту получены зашумленные данные (объект *s* типа Series) по его транзакциям: `s = pd.Series(data=['10', 20, 30.5, 'hello', 40, -50, 60.2, 'world', 70, 80], index=range(2, 22, 2))`
Выберите из *s* только целочисленные элементы (имеющие тип `int`) и вычислите их среднее арифметическое. Ответ округлите до одного знака после запятой.
2. Считайте в переменные *tr_mcc_codes*, *tr_types*, *transactions* и *gender_train* из одноимённых таблиц из папки *data*. Для таблицы *transactions* используйте только первые *n=1000000* строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Рассчитайте медиану суммы транзакций по полю *amount* из таблицы *transactions* для тех строк, в которых значение *mcc_code* начинается с символа '1' (т.е. первый символ строки – '1'). Перед расчётом удалите дублирующиеся строки по столбцам [*mcc_code*, *tr_type*], оставляя только последние из дублирующихся (`keep='last'`). Выведите полученную медиану с округлением до двух знаков.

Вариант №31

1. Сгенерируйте Series из 200 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами), используя `np.random.seed(242)`. Возведите каждое значение серии в квадрат, а значения индекса увеличьте в 4 раза. Выведите количество элементов, значения которых строго больше 5.
2. Считайте в переменные `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. В датафрейме `transactions` отсортируйте данные по убыванию суммы транзакций (`amount`), возьмите первые 10 строк (т.е. топ-10 по величине суммы) и вычислите их среднее значение `amount`. Ответ округлите до двух знаков после запятой.

Вариант №32

1. Сгенерируйте Series из 100 значений нормально распределённой случайной величины (`np.random.normal` с дефолтными параметрами – нулевым средним и единичной дисперсией), используя `np.random.seed(242)`. Умножьте каждое значение серии на 2, а значения индекса – на 3. Выведите сумму элементов, у которых индекс кратен 4 (т.е. остаток от деления на 4 равен 0).

Пояснение: фиксированное значение `seed` нужно, чтобы ответы у всех выполняющих это задание были одинаковые и их можно было проверить (так как генерируются одинаковые series).

2. Считайте в переменные `transactions` и `gender_train` из одноимённых таблиц из папки `data`. Для таблицы `transactions` используйте только первые `n=1000000` строк. Обратите внимание на разделители внутри каждого из файлов – они могут различаться. Проверьте содержимое созданных датафреймов.
3. Для каждого уникального значения `tr_type` в датафрейме `transactions` вычислите среднюю сумму транзакций (`amount`). Отсортируйте полученные средние по убыванию и выведите первые три значения (т.е. топ-3 типа транзакций по среднему `amount`) вместе с соответствующими `tr_type`.

Описание данных

Таблица `transactions.csv`

Описание

Таблица содержит историю транзакций клиентов банка за один год и три месяца.

Формат данных

```
customer_id,tr_datetime,mcc_code,tr_type,amount,term_id
111111,15 01:40:52,1111,1000,-5224,111111
111112,15 15:18:32,3333,2000,-100,11122233
...
```

Описание полей

- `customer_id` — идентификатор клиента;
- `tr_datetime` — день и время совершения транзакции (дни нумеруются с начала данных);
- `mcc_code` — мсс-код транзакции;
- `tr_type` — тип транзакции;
- `amount` — сумма транзакции в условных единицах со знаком; `+` — начисление средств клиенту (приходная транзакция), `-` — списание средств (расходная транзакция);
- `term_id` — идентификатор терминала;

Таблица `gender_train.csv`

Описание

Данная таблица содержит информацию по полу для части клиентов, для которых он известен. Для остальных клиентов пол неизвестен.

Формат данных

```
customer_id,gender
111111,0
111112,1
...
```

Описание полей

- `customer_id` — идентификатор клиента;
- `gender` — пол клиента;

Таблица `tr_mcc_codes.csv`

Описание

Данная таблица содержит описание мсс-кодов транзакций.

Формат данных

```
mcc_code;mcc_description
1000;словесное описание мсс-кода 1000
2000;словесное описание мсс-кода 2000
...
```

Описание полей

- `mcc_code` — мсс-код транзакции;
- `mcc_description` — описание мсс-кода транзакции.

Таблица `tr_types.csv`

Описание

Данная таблица содержит описание типов транзакций.

Формат данных

```
tr_type;tr_description
1000;словесное описание типа транзакции 1000
2000;словесное описание типа транзакции 2000
...
```

Описание полей

- `tr_type` — тип транзакции;
- `tr_description` — описание типа транзакции;

Ссылка для скачивания файлов:

<https://cloud.mail.ru/public/L2DU/UNuECvB1g>